

# **Fault-Frequency-Agnostic Checkpointing Strategies**

©2026

**Joseph Vinduska**

Submitted to the graduate degree program in Department of Electrical Engineering and Computer Science and the Graduate Faculty of the University of Kansas in partial fulfillment of the requirements for the degree of Master of Science.

Committee members

---

Dr. Hongyang Sun, Chair

---

Dr. Arvin Agah, Member

---

Dr. Drew Davidson, Member

Date defended: 05/06/2026

The Thesis Committee for Joseph Vinduska certifies  
that this is the approved version of the following thesis:

Fault-Frequency-Agnostic Checkpointing Strategies

---

Dr. Hongyang Sun, Chair

Date approved: 05/06/2026

# Abstract

Checkpointing strategies in high-performance computing traditionally employ the Young-Daly formula to determine the (first-order) optimal duration between checkpoints, which assumes a known mean time between faults (MTBF). However, in practice, the MTBF may not be known accurately or may vary, causing Young-Daly checkpointing to perform sub-optimally. In 2021, Sigdel et al. introduced the CHORE (CHeckpointing Overhead and Rework Equated) checkpointing strategy, which is MTBF-agnostic yet demonstrates a bounded increase in overhead compared to the optimal strategy. This thesis analyzes and extends the CHORE framework in several ways. First, it verifies Sigdel et al.'s claims about the relative overhead of the CHORE strategy through both event-driven simulations and expected runtimes derived from the underlying probabilistic model. Second, it extends the CHORE strategy to silent errors, which must be deliberately checked for to be detected. In this scenario, the overhead compared to optimal checkpointing is once more analyzed through simulations and expected runtimes. Third, a rollback heuristic is proposed to offer improved performance of the CHORE algorithm under typical runtime scenarios by interpreting CHORE as an additive-increase multiplicative-decrease model and tuning the parameters.

## **Acknowledgements**

I would like to thank my advisor, Dr. Hongyang Sun, for his time and insight as I researched and wrote this thesis. I would like to thank Redouane Elghazi for being part of our meetings and providing valuable feedback. I would also like to thank my professors from my undergraduate program at Temple University, especially Jeffrey Zheng, who taught me much of the probability and statistics employed in portions of this analysis. Finally, I would like to thank the other members of my committee, Dr. Arvin Agah and Dr. Drew Davidson, for their commitment to learning and the guidance they provided during my time at the Univeristy of Kansas.

# Contents

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                     | <b>8</b>  |
| 1.1      | Overview . . . . .                      | 8         |
| 1.2      | Contributions . . . . .                 | 9         |
| 1.3      | Organization . . . . .                  | 9         |
| <b>2</b> | <b>Related Work</b>                     | <b>11</b> |
| 2.1      | Young-Daly Checkpointing . . . . .      | 11        |
| 2.2      | The CHORE Algorithm . . . . .           | 11        |
| 2.3      | Other Algorithms . . . . .              | 13        |
| <b>3</b> | <b>CHORE and Variants</b>               | <b>15</b> |
| 3.1      | Models and Assumptions . . . . .        | 15        |
| 3.2      | CHORE Scenario . . . . .                | 16        |
| 3.3      | Analysis for Fail-Stop Faults . . . . . | 18        |
| 3.4      | Analysis for Silent Errors . . . . .    | 19        |
| <b>4</b> | <b>Rollback Algorithm</b>               | <b>22</b> |
| 4.1      | Adaptation . . . . .                    | 22        |
| 4.2      | Generalization . . . . .                | 22        |
| 4.3      | Optimal Rollback Factor . . . . .       | 24        |
| <b>5</b> | <b>Experimental Results</b>             | <b>26</b> |
| 5.1      | Setup . . . . .                         | 26        |
| 5.2      | Observations on Sigdel et al. . . . .   | 26        |

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| 5.3      | Runtime Scenarios . . . . .                                          | 27        |
| 5.4      | Rollback Strategies . . . . .                                        | 29        |
| <b>6</b> | <b>Conclusion</b>                                                    | <b>35</b> |
| 6.1      | Findings . . . . .                                                   | 35        |
| 6.2      | Future Work . . . . .                                                | 36        |
| <b>A</b> | <b>Expected Runtime Derivation</b>                                   | <b>39</b> |
| A.1      | Preliminaries . . . . .                                              | 39        |
| A.2      | Fail-Stop Faults, Faults During Checkpointing and Recovery . . . . . | 39        |
| A.2.1    | Young-Daly . . . . .                                                 | 40        |
| A.2.2    | CHORE . . . . .                                                      | 41        |
| A.3      | Fail-Stop Faults, Safe Checkpointing and Recovery . . . . .          | 43        |
| A.3.1    | Young-Daly . . . . .                                                 | 44        |
| A.3.2    | CHORE . . . . .                                                      | 44        |
| A.4      | Silent Faults, Safe Checkpointing and Recovery . . . . .             | 45        |
| A.4.1    | Young-Daly . . . . .                                                 | 45        |
| A.4.2    | CHORE . . . . .                                                      | 46        |
| <b>B</b> | <b>Interval Interpolation</b>                                        | <b>48</b> |
| B.1      | Fail-Stop Faults . . . . .                                           | 48        |
| B.2      | Silent Faults . . . . .                                              | 49        |

## List of Figures

|     |                                                                                                                                                       |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Assumed ideal interval spacing . . . . .                                                                                                              | 24 |
| 5.1 | Overhead in terms of the minimum required work (simulated; $w = 100\mu$ ). . . . .                                                                    | 27 |
| 5.2 | Extension of Figure 5.1, CHORE and Young-Daly efficiency for large $c/\mu$ . . . . .                                                                  | 28 |
| 5.3 | CHORE overhead / Young-Daly overhead under Sigdel et al.'s assumptions (fail-stop faults; faults possible during checkpointing and recovery). . . . . | 29 |
| 5.4 | CHORE overhead / Young-Daly overhead for fail-stop faults, no faults during checkpointing or recovery. . . . .                                        | 30 |
| 5.5 | CHORE overhead / Young-Daly overhead for silent faults, no faults during checkpointing or recovery. . . . .                                           | 30 |
| 5.6 | Overhead relative to baseline CHORE for various Rollback coefficients; fail-stop faults, faults possible during checkpointing and recovery. . . . .   | 32 |
| 5.7 | Overhead relative to CHORE for various Rollback coefficients; fail-stop faults, no faults during checkpointing or recovery. . . . .                   | 33 |
| 5.8 | Overhead relative to CHORE for various Rollback coefficients; silent faults, no faults during checkpointing or recovery. . . . .                      | 34 |

# Chapter 1

## Introduction

### 1.1 Overview

In high-performance computing (HPC), the sheer number of processors and other components means that HPC systems encounter faults relatively often, while the user of a typical PC often goes months at a time without a complete system failure. To mitigate loss from faults, various checkpointing strategies are employed; perhaps the most well-known checkpointing strategy comes from the Young-Daly equation for (first-order) optimal checkpointing. We classify faults into two types: *fail-stop faults*, which are immediately detected, such as the failure of a CPU or hard drive; and *silent faults*, such as data corruption, which must deliberately be checked for. Variations of the Young-Daly equation exist for both.

As with any model, the Young-Daly equation relies on certain assumptions. Namely, faults are independent and exponentially distributed, forming a Poisson distribution with some mean time between faults (MTBF) of  $\mu$ ; we also assume checkpointing takes some fixed time  $c$ . However, these assumptions may not always hold. Some research finds a Weibull distribution or even a gamma distribution to be a better model of fault occurrences [6] (however, [10] notes that a Weibull distribution reduces the job execution time, but does not significantly impact the optimal checkpointing interval). More importantly for this analysis, the exact MTBF of a system may not be known, or it may vary during the execution of a job; Sigdel et al. cite a number of studies examining various causes for this, including system load [5], time of day and day of the week [6]; and particular components used for a job, e.g. GPUs with double-bit-error detection but not correction [9].

To address this uncertainty, Sigdel et al. [8] proposed the CHORE checkpointing strategy in

2021. This strategy does not require knowledge (or an estimate) of the MTBF, yet demonstrates a bounded increase in overhead compared to the Young-Daly (YD) checkpointing strategy. Sigdel et al. find that, the ratio of CHORE’s overhead to Young-Daly’s overhead is bounded; for what this paper will call the *overhead percentage ratio*, Sigdel et al. find the maximum to be 1.26.

In Chapter 5, we verify the claims of Sigdel et al. through event-driven simulations and models of expected runtime, and largely agree with the authors’ results. On the way, we extend the logic of the CHORE algorithm into the domain of silent errors, deriving a similar ratio both experimentally and theoretically. Finally, we propose a heuristic to reduce the overhead of CHORE in standard HPC environments.

## 1.2 Contributions

The main contributions of this work are as follows:

- We examine Sigdel et al.’s claims about the overhead of CHORE via simulations and novel mathematical models, and find results consistent with the authors’.
- We extend the logic of the standard CHORE algorithm (for fail-stop errors) to silent errors, and find it performs comparably. Again, these findings are supported both by simulations and mathematical models.
- We propose heuristics to reduce the overhead of the CHORE algorithm, and find that in simulations, they shrink overhead by approximately 10 – 15%, depending on the assumptions on types of failure (silent or fail-stop) and which, if any, segments of execution are free of faults.

## 1.3 Organization

The remainder of this work is structured as follows. Chapter 2 considers related work: the classic Young-Daly checkpointing strategy; the CHORE checkpointing strategy that serves as the pri-

mary focus of our analysis; and other checkpointing strategies that address the same challenges as CHORE does. In Chapter 3, we analyze the assumptions used in [8] for CHORE’s debut, extend the reasoning of the CHORE algorithm to develop variants for silent errors and restricted fault possibilities, and present equations for the expected checkpointing, loss, recovery, and total overhead in each scenario. In Chapter 4, we develop the Rollback variant of the CHORE algorithm (applicable to all scenarios), and estimate the theoretically optimal rollback coefficient. Chapter 5 begins with some observations on the work of Sigdel et al., re-deriving the ratio of CHORE overhead to Young-Daly overhead, before repeating this process for the novel scenarios discussed in Chapter 3. We then discuss the effectiveness of the Rollback strategy, using event-driven simulations to measure the improvement and empirically determine the most effective rollback coefficients. Finally, Chapter 6 summarizes our findings and discusses several directions in which our work could be extended.

## Chapter 2

### Related Work

#### 2.1 Young-Daly Checkpointing

Perhaps the most well-known checkpointing strategy is Young-Daly checkpointing [11], [2]. This model is the first-order approximation of optimal checkpointing for independent, exponentially-distributed fail-stop faults. The process is simple: do work for  $\sqrt{2\mu c}$  units of time, then checkpoint. It is important to note that this strategy assumes that no faults will happen during a checkpoint. When checkpoint faults are possible, the optimal strategy can be approximated by doing  $\sqrt{2\mu c} - c$  work before checkpointing, as Sigdel et al. use. However, this is a less effective approximation. Notice that (in the admittedly unreasonable case) when  $c = 2\mu$ , this formula implies the optimal strategy is to do no work and only checkpoint. Fortunately, it is reasonable to assume that the checkpointing cost is much smaller than the MTBF, so this approximation works in practice.

Young-Daly checkpointing can also be derived for silent errors. In this case, the optimal time between checkpoints becomes  $\sqrt{\mu c}$  units of time, as described by Benoit et al. [1].

#### 2.2 The CHORE Algorithm

Introduced by Sigdel et al., the CHORE algorithm (CHeckpointing Overhead and Rework Equated) is based on the observation that during any particular sequence, Young-Daly checkpointing has an expected rework time equal to the total expended checkpointing time [8]. From this realization, the authors devise a simple checkpointing strategy to emulate this property.

Let a *run* be the period of work from the start of the process (or immediately following a fault)

until the next fault (or the end of the process.) We then divide the run into *intervals*, with each interval being a segment of work followed by a checkpoint. We will number each interval with the first interval being 0, for reasons explained momentarily. The CHORE algorithm can then be stated by defining the duration of interval number  $i$  as such:

$$dur(i) = \begin{cases} 2c & i = 0 \\ 2ic & i \geq 1 \end{cases}$$

That is, the algorithm starts with an interval of size  $2c$ , doing  $1c$  of work and then checkpointing; the next interval is the same, and following intervals increase the amount of work done by  $2c$  each interval. When a fault occurs, the process starts again at interval 0. We begin counting the first interval at 0 instead of 1 because it does not follow the pattern of increasing interval sizes; in fact, it serves to bootstrap the sequence by providing the first section with expected loss.

Sigdel et al. perform a more complicated derivation with Markov chains, but notice that under the assumption  $c \ll \mu$ , we can treat an interval as having uniform probability of a fault. Then the expected rework time during an interval (given a fault occurs) is simply half the work done in the interval. Interval 0 serves to bootstrap the process. At interval 1, the total time spent checkpointing is  $c$ ; to make the expected loss (and thus rework) equal  $c$ , interval 1 must have duration of  $2c$  under our uniform approximation. We see growth beginning in the next interval: as the aggregate checkpointing has increased by  $c$ , the size of the interval must increase by  $2c$  (increasing the expected loss by  $c$ ) to retain the desired equivalence.

One important fact about the CHORE algorithm is that it is MTBF-agnostic, as intervals depends solely on  $c$ . Sigdel et al. suggest that CHORE may be more effective than Young-Daly checkpointing in practice; they cite many studies that show MTBF estimates can be quite inaccurate, and that MTBF can change during the course of a job based on hardware and environmental conditions.

Additionally, the downsides of CHORE are relatively limited. While, for a constant MTBF, it is not as effective as Young-Daly checkpointing, the authors demonstrate that the performance of CHORE is within a fixed factor of Young-Daly performance. The authors claim that CHORE

has less than 26% additional overhead compared to Young-Daly checkpointing under the standard assumptions. While there is some ambiguity in Sigdel et al.’s definition of overhead used for this claim (as discussed in Section 5.3), we agree that overhead is bounded relative to Young-Daly checkpointing.

## 2.3 Other Algorithms

There are several approaches to deal with unknown or variable MTBF, although they are not typically entirely MTBF-agnostic. The general approach is to estimate the MTBF from faults encountered during a job, perhaps in tandem with an estimated initial MTBF. For example, Tiwari et al. [10] describes the simple technique of using a rolling average of the observed MTBF; El-Sayed and Schroeder [3], [4] also suggests linearly or exponentially weighted moving averages, which can better capture spikes in fault frequency, as system age or environmental conditions may induce. Algorithms that are dynamically influenced by fault frequency are not uncommon; Skip Checkpointing [10] dynamically responds to faults, and the weighted average techniques in [3], [4] show performance similar to, or sometimes exceeding, that of Young-Daly checkpointing. Furthermore, Sigdel et al. even propose a CHORE variant, En-CHORE, that uses an estimated MTBF to select more effective interval durations.

These strategies also address that MTBF may vary during the course of a job, or that faults may not exactly follow a Poisson distribution. For example, analysis of HPC logs by Tiwari et al. [10] finds that, in practice, faults tend to exhibit clustering; or in other words, given a fault has recently occurred, another imminent fault is more likely than a Poisson distribution would suggest. From this observation, the authors propose two techniques: *Lazy Checkpointing*, where the checkpointing interval is dynamically increased as a job continues without a fault, generalizing the underlying exponential distribution to a Weibull distribution; and *Skip Checkpointing*, which occasionally skips a checkpoint when sufficient time since a fault has passed, and the risk is relatively low. Similarly, the moving averages used in [3], [4] can capture variations in system performance, although effectiveness varies based on weighting parameters and window size.

To the best of our knowledge, CHORE is the only published checkpointing strategy that does not even use an estimate of the MTBF, excluding the “crude rules-of-thumb such as ‘checkpoint once every hour’” mentioned in [3]. There are several possible explanations for this. Hardware manufactures provide estimates of component reliability, and many studies, including Schroeder and Gibson [7], perform further analysis and account for factors such as age, so MTBF estimates are readily available. However, as Sigdel et al. draw our attention to, the findings in [7] indicate the MTBF of disk drives is, at times can be off by as much as a factor of 30. However, if we assume published reliability is usually fairly accurate, factors described in [3] may also explain why fully MTBF-independent checkpointing has not received much attention. Using real job traces, El-Sayed and Schroeder find that Young-Daly checkpointing with an estimated MTBF (and assumed accurate checkpointing cost) has no more than 10% wasted work compared to using the true MTBF— even when the estimate of  $\mu$  is off by as much as a factor of 2. In practice, MTBF estimates can still be efficient even with a fair quantity of error.

## Chapter 3

### CHORE and Variants

#### 3.1 Models and Assumptions

We begin by considering Sigdel et. al.'s CHORE scenario, and adapting it to other sets of assumptions often used in checkpointing analysis.

As described in the previous chapter, CHORE makes the following assumptions:

- Faults are independent and exponentially distributed, forming a Poisson distribution;
- Only fail-stop faults occur ;
- Checkpointing and recovery require the same fixed length of time;
- It is possible for faults to occur at any point during the job, including checkpointing or restoration.

We can then begin our analysis with these assumptions. As an alternative to the Markov chain model employed by Sigdel et al. [8], we can calculate the expected time to completion of a job under CHORE or Young-Daly checkpointing via the expected values of the underlying fault distribution.

To begin, Let  $T$  be the runtime of the job,  $w$  the required work,  $C$  the total time spent on successful checkpoints,  $R$  the total time on successful restorations, and  $L$  the total lost work. Similarly, let  $\bar{C}$  be the average successful checkpointing per fault,  $\bar{R}$  the average successful restoration time per fault,  $\bar{L}$  the average lost work per fault, and  $F$  the number of faults encountered during a run.

For any process,

$$T = w + L + C + R \quad (3.1)$$

Taking averages with respect to any specific run,

$$T = w + F \cdot \bar{L} + F \cdot \bar{C} + F \cdot \bar{R} \quad (3.2)$$

In a long-running process, by definition,  $E[F] = T/\mu$ . Similarly,  $\bar{L} = E[\bar{L}]$ ,  $\bar{C} = E[\bar{C}]$ , and  $\bar{R} = E[\bar{R}]$ . All of these values can be calculated for our various models.

For simplicity, we will make two further assumptions. First, all faults may happen during recovery or an interval (which will lead to slightly underestimating the first loss, which must happen during an interval). Second, we assume that all recoveries have a chance of failure; however, for a process to complete, the final recovery must not experience a fault (slightly underestimating the cost of the final recovery). While making these assumptions introduces some error, it is a fixed amount; for a sufficiently long process, it becomes an arbitrarily small portion of the calculated runtime.

### 3.2 CHORE Scenario

Under the assumptions and model in the previous section, every fault that occurs is detected; then, for a sufficiently long process, isolating  $T$  gives

$$E[T] = \frac{w}{(1 - \frac{E[\bar{L}] + E[\bar{C}] + E[\bar{R}]}{\mu})} \quad (3.3)$$

It is worth noting that the  $F \cdot \bar{R}$  portion of the equation will not change between strategies, as we have a fixed restoration time and a restoration must be attempted after each fault. Thus,  $E[\bar{R}]$  is simply the odds of completing the restoration. From this,

$$E[\bar{R}] = (1 - e^{-r/\mu}) \quad (3.4)$$

for all checkpointing algorithms.

As we produce the terms to fill this equation, you may notice that some terms have a factor of  $c$  or  $r$ . However, in the final result, they appear in a fraction with  $\mu$ . Multiplying the top and bottom of this fraction by  $\frac{1}{c} = \frac{1}{r}$  eliminates these terms and transforms  $\mu$  to  $\frac{\mu}{c}$ , leaving an expression with only one independent variable,  $\frac{c}{\mu}$  (or its inverse,  $\frac{\mu}{c}$ ). Notice that this results in an equation  $E[T] = w \cdot s$ , or the work  $w$  times a scaling factor. Thus, the overhead ratio for to jobs with the same base work only depends on  $\frac{\mu}{c}$  and no other variables.

Under this same set of assumptions, the equations for Young-Daly (YD) checkpointing are straightforward to calculate; we find

$$E[\bar{L}_{YD}] = (1 - e^{-r/\mu}) \cdot \left( \frac{\mu - re^{-r/\mu} - \mu e^{-r/\mu}}{1 - e^{-r/\mu}} \right) + e^{-r/\mu} \cdot \frac{\mu - \sqrt{2\mu c} \cdot e^{-\sqrt{2\mu c}/\mu} - \mu e^{-\sqrt{2\mu c}/\mu}}{1 - e^{-\sqrt{2\mu c}/\mu}} \quad (3.5)$$

and

$$E[\bar{C}_{YD}] = e^{-\sqrt{2c/\mu}} \cdot \frac{1}{1 - e^{-\sqrt{2c/\mu}}} \cdot c \quad (3.6)$$

For CHORE (CH), we find that

$$E[\bar{L}_{CH}] = (1 - e^{-r/\mu}) \cdot \left( \frac{\mu - re^{-r/\mu} - \mu e^{-r/\mu}}{1 - e^{-r/\mu}} \right) + e^{-r/\mu} \cdot \left( \mu - (2ce^{-2c/\mu} + \sum_{n=1}^{\infty} 2nc \cdot e^{-c(n^2+n+2)/\mu}) \right) \quad (3.7)$$

and

$$E[\bar{C}_{CH}] = c \cdot \sum_{n=0}^{\infty} e^{-c(n^2+n+2)/\mu} \quad (3.8)$$

The full derivation of this, and all other expected values, can be found in Appendix A.

Notice that while these terms (and some that follow) contain infinite sums, they can easily be calculated to arbitrary precision. The exponential factor, which dominates, grows at a polynomial rate, making the values less than a (converging) geometric series with the same base (or the base

times  $\frac{1}{n}$  when truncated after term  $n$ ). Once several terms have been calculated, it is possible to derive limits on the error. In practice, we found it was effective to calculate these sums to the limit of double-precision floating-point values. This level of accuracy did not seem to negatively impact the calculations; expected runtime is within a few parts per thousand of the average simulated runtime (for jobs with sufficient duration), with this error attributed primarily to statistical noise. Similar verification was performed on the equations in the following sections, yielding comparable accuracy.

### 3.3 Analysis for Fail-Stop Faults

For our first alternative scenario, we begin by considering the classic Young-Daly checkpointing assumptions: no faults occur during checkpointing or recovery. We allow all other assumptions to hold, including matching time for checkpointing and recovery.

Since some segments of the job are now immune to errors, we must make a minor adjustment to the algorithm to preserve its aggregate checkpointing and expected loss parity. Since  $c$  of every interval no longer has the chance of a fault, this change is as simple as adding another  $c$  of work to each interval; this gives interval sizes

$$dur(i) = \begin{cases} 3c & i = 0 \\ (2i + 1)c & i \geq 1 \end{cases}$$

We must also adapt our runtime estimation equation, since restoration always succeeds, and any checkpoint that begins will successfully complete. Importantly, we have to be careful in how we calculate the expected number of faults, as faults can only happen during the "work" portion of the job. Since faults occur after an average period of  $\mu$ , we find  $F = \frac{T - F(E[CH] + E[R])}{\mu}$ . Solving for  $F$ , substituting into our original equation, and solving for  $T$  gives the following:

$$E[T] = \frac{w}{1 - \frac{E[C] + E[L] + E[R]}{\mu + E[C] + E[R]}} \quad (3.9)$$

Again, both models have the same expected recovery:

$$E[\bar{R}_{YD}] = E[\bar{R}_{CH}] = r \quad (3.10)$$

We then find that for Young-Daly checkpointing,

$$E[\bar{L}_{YD}] = \frac{\mu - \sqrt{2\mu c} \cdot e^{-\sqrt{2c/\mu}} - \mu e^{-\sqrt{2c/\mu}}}{1 - e^{-\sqrt{2c/\mu}}} \quad (3.11)$$

$$E[\bar{C}_{YD}] = c \cdot e^{-\sqrt{2c/\mu}} \cdot \frac{1}{1 - e^{-\sqrt{2\mu c}/\mu}} \quad (3.12)$$

For CHORE, we can derive

$$E[\bar{L}_{CH}] = \mu - (2ce^{-2c/\mu} + \sum_{n=0}^{\infty} 2nc \cdot e^{-c(n^2+n+2)/\mu}) \quad (3.13)$$

$$E[\bar{C}_{CH}] = c \cdot \sum_{n=0}^{\infty} e^{-c(n^2+n+2)/\mu} \quad (3.14)$$

Again, the full derivation of these equations is listed in Appendix A.

### 3.4 Analysis for Silent Errors

We also consider adapting the CHORE algorithm to handle silent errors. For simplicity, we assume the validation step is performed immediately before a checkpoint is attempted and takes a negligible amount of time. This means no checkpoint will be attempted if an error occurred during the preceding portion of the interval. As with the previous scenario, we assume that there are no faults during the checkpointing or recovery phases of the job.

Extending the rationale behind the CHORE algorithm described in Chapter 2, we find that for silent errors, CHORE should use the following interval durations:

$$dur(i) = \begin{cases} 2c & i = 0 \\ (i+1)c & i \geq 1 \end{cases}$$

Or in other words, intervals with half the work of the corresponding fail-stop error interval. As with the initial model, we want the expected loss to equal the aggregate checkpointing time. After bootstrapping with interval 0, our aggregate checkpointing time is  $c$ ; if a loss occurs, we will not detect it until the end of the work segment, and thus lose the full segment of work. Hence, for interval 1, we should work for  $c$  before the checkpoint. In interval 2, aggregate checkpointing is  $2c$ , so the quantity of work grows from  $c$  to  $2c$ , and so on in the following intervals.

Before we provide our new calculation for expected runtimes, we need to make one additional change to our base model. Since silent errors must be checked for, and we only check after discrete intervals, it is possible for multiple errors to occur during an interval, but there will only be a single corresponding detection. Recall the wording of our initial formulation of expected runtime:  $E[\overline{CH}]$ ,  $E[\overline{L}]$ , and  $E[\overline{R}]$ , are averages per *encountered* fault. Thus, the only change we need to make to our model is replacing the expected number of faults with the expected number of *encountered* faults. We will do so by multiplying the expected number of faults by the expected faults detected ( $D$ ) per fault,  $\frac{D}{F}$ , as the inverse of this factor is straightforward to calculate. After applying the  $\frac{D}{F}$  factor to our equation, we follow the same adjustment for fault-possible time as used in the previous section. We then find our model is

$$E[T] = \frac{w}{1 - \frac{E[\frac{D}{F}](E[C]+E[L]+E[R])}{\mu + E[\frac{D}{F}](E[C]+E[R])}} \quad (3.15)$$

Fortunately, there is some simplification to this scenario as well. Restoration costs remain  $r$ , since restoration always succeeds; and since a fault causes the entire interval to be lost, there is no fractional component in the Young-Daly model.

Before we present the components for each model, let us introduce a helper function  $\lambda(x)$ , which is the expected number of faults during period  $x$  given that at least one fault occurs. Then, we find

$$\lambda(x) = \frac{x/\mu}{1 - e^{-x/\mu}} \quad (3.16)$$

We will derive the expected runtime for CHORE for silent errors and Young-Daly checkpointing for silent errors, as described in Chapter 2. For the Young-Daly model, we derive the following

terms:

$$E[\bar{L}_{YD}] = \sqrt{\mu c} \quad (3.17)$$

$$E[\bar{R}_{YD}] = r \quad (3.18)$$

$$E[\bar{C}_{YD}] = c \cdot e^{-\sqrt{c/\mu}} \cdot \frac{1}{1 - e^{-\sqrt{\mu c}/\mu}} \quad (3.19)$$

$$E\left[\frac{F}{D}\right] = \lambda(\sqrt{\mu c}) \quad (3.20)$$

And for silent CHORE,

$$E[\bar{L}_{CH}] = c(1 - e^{-c/\mu}) + c \left( \sum_{n=0}^{\infty} e^{-(\frac{n^2+n}{2}+1)c/\mu} \right) \quad (3.21)$$

$$E[\bar{R}_{YD}] = r \quad (3.22)$$

$$E[\bar{C}_{CH}] = c \sum_{n=0}^{\infty} e^{-(\frac{n^2+n}{2}+1)c/\mu} \quad (3.23)$$

$$E\left[\frac{F}{D}\right] = (1 - e^{-c/\mu}) \cdot \lambda(c) \quad (3.24)$$

$$+ \sum_{n=0}^{\infty} (e^{-(c\frac{n^2+n+2}{2})/\mu} - e^{-c(\frac{(n+1)^2+(n+1)+2}{2})} \cdot \lambda((n+1)c)/\mu) \quad (3.25)$$

Once more, the full derivation is listed in Appendix A.

## Chapter 4

### Rollback Algorithm

#### 4.1 Adaptation

We investigate heuristics for reducing the overhead of the CHORE algorithm, both in its initial form and our alternative scenarios.

Our key observation is that we assume  $c \ll \mu$ , which means we make too many checkpoints at the beginning of a sequence. To mitigate this, our goal was to devise a "rollback" strategy: when a fault occurs, "roll back" to an earlier state of the algorithm, presumably with an interval size larger than one would get from restarting the sequence from the beginning. A variety of transformations were considered; ultimately, scaling the interval size at the time of a fault by a linear factor was selected due to its simplicity and effectiveness. This provides the additional benefit of considering the algorithm as an additive-increase, multiplicative-decrease model. The variations of CHORE considered thus far operate on the interval duration with an additive increase of  $2c$  (fail-stop faults) or  $c$  (silent faults) and have a multiplicative decrease of 0; that is, the sequence resets completely. It is reasonable to expect that these parameters could be tuned to give better performance. While Sigdel et al.'s justification of balancing the expected loss and aggregate checkpointing does not strictly hold in the Rollback model, we will not adjust the additive increase, and will focus on tuning the multiplicative decrease in this scenario.

#### 4.2 Generalization

Adapting the CHORE algorithm to a rollback strategy presented several questions and possibilities.

First, consider the original CHORE scenario, where interval sizes are always multiples of  $2c$ . As a generalization of the CHORE algorithm, we allowed rollback to start sequences with interval sizes that would not occur in the base CHORE model (i.e. fractional multiples of  $c$ ), which gave a significant improvement in Rollback performance.

Similarly, consider how the interval sizes grow in the original CHORE scenario. Excluding interval 0, subsequent intervals grow by  $2c$ , but is this a discrete jump? If so, that would suggest that a fault at time  $t$  and a fault at  $t + \varepsilon$  may roll back to different locations. Noticing the total runtime at the end of an interval is the sum the duration of all previous intervals, we find that  $t = 2c + 2c(1 + 2 + 3 + \dots + n)$  for the total runtime at the end of interval  $n$ ; similar results can be obtained for the other scenarios we propose. Of course, the inner sum is the well-known identity  $1 + 2 + \dots + n = \frac{n(n+1)}{2}$ , and with this, we can solve for  $n$ , giving us a continuous function for interval size in terms of total runtime. In fact, this equation can be further generalized for non-zero starting intervals, allowing it to be used in our rollback scenario.

While we leave the full derivation of these equations to Appendix B, let us note the following equations. In a sequence with (fault-possible) runtime (in terms of  $c$ )  $tc$  and starting interval size (also in terms of  $c$ )  $zc$ ; we find that for fail-stop faults, we are at the end of interval

$$n = \frac{-(z+1) + \sqrt{(z-1)^2 + 4t}}{2} \quad (4.1)$$

and at the start of a new interval number  $n + 1$  with duration  $z + 2nc$

Similarly, for silent errors, after running for  $tc$  (fault-possible time) after starting with interval size  $zc$

$$n = -(z - 1/2) + \sqrt{(z - 1/2)^2 + 2t} \quad (4.2)$$

and we can consider this point the start of interval  $n + 1$  with duration  $z + nc$ .

We found that using the continuous interval size model made a small but persistent improvement to the rollback efficiency; additionally, we found it a logical extension to the CHORE model. For these reasons, we conducted most rollback experiments under the continuous interval growth

assumption.

### 4.3 Optimal Rollback Factor

As we will discuss in Section 5, we found the optimal rollback factor varies based on the scenario used. However, we propose a theoretical model that approximates the optimal rollback factor under all scenarios. In this derivation, we assume continuous interval growth, as mentioned above; we also will not repeat the first interval after a rollback, and we do not limit the minimum size an interval can roll back to. These potential variations on these parameters are discussed in more detail in Section 5.4.

To begin our derivation, we assume that Young-Daly checkpointing is optimal, so we expect CHORE's growth strategy to be near-optimal when the average (fault-possible) interval size is  $\sqrt{2\mu c}$ . We also know that, on average, faults occur after  $\mu$  time. Thus, our estimate of the ideal sequence is a number intervals with size  $\sqrt{2\mu c}$  in the center; with intervals decreasing by  $2c$  before and increasing by  $2c$  after.

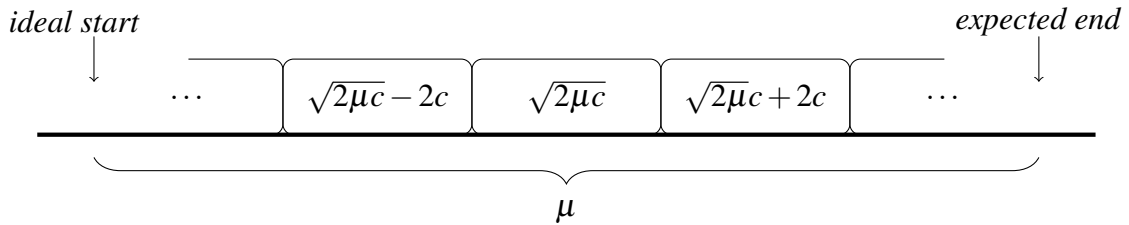


Figure 4.1: Assumed ideal interval spacing

Since we want the average interval size to be  $\sqrt{2\mu c}$ , we must have an equal number of smaller and larger intervals since we have a linear rate of change. Call the number of larger intervals (and thus the number of smaller intervals)  $x$ , so that we have  $2x + 1$  intervals in this run of duration  $\mu$ . Additionally, notice that the number of intervals is the total length of the sequence over the average interval size, or  $2x + 1 = \frac{\mu}{\sqrt{2\mu c}}$ . The first interval in this sequence will have duration  $\sqrt{2\mu c} - 2cx$  and the final interval  $\sqrt{2\mu c} + 2cx$ . Thus, the optimal rollback factor is the length of the start interval

over the length of the final interval, or

$$\frac{\sqrt{2\mu c} - 2xc}{\sqrt{2\mu c} + 2xc} \quad (4.3)$$

Since  $2x \approx 2x + 1 = \frac{\mu}{\sqrt{2\mu c}}$ , we can substitute this value. Simplification then yields an optimal rollback factor of  $\frac{1}{3}$ .

While this model serves as a decent approximation, it is important to acknowledge some of the simplifications it makes. Of course, an interval size of  $\sqrt{2\mu c}$  is not guaranteed to exist; the intervals surrounding this Young-Daly interval will likely not add up to exactly  $\mu$ . Perhaps the most significant assumption is more subtle. We assume that a fault will occur after  $\mu$  units of runtime, which is correct, as  $\mu$  is the expected value of the exponential distribution that faults are drawn from. What is not correct, however, is to equate the expected time of a fault to the expected *interval size* at the time of a fault. The expected value of the time of the fault is derived by taking  $\int_0^\infty xe^{-x/\mu} dx$ , that is, the integral of the time multiplied with the exponential probability distribution. However, to calculate the expected interval size when a fault occurs, we must use a different equation. As discussed in the previous section, the interval size is approximately the square root of the interval's duration. Thus, the expected interval size when a fault occurs is better represented by  $\int_0^\infty a\sqrt{xe^{-x/\mu}} dx$ , which cannot be linearly transformed into the expected time of a fault. Nonetheless, this model gives us a starting point for finding the optimal rollback factor, and as the following section discusses,  $\frac{1}{3}$  turns out to perform quite well in several scenarios.

We can make one more observation. This derivation only depends on an ideal interval size, a constant rate of change, and faults with a fixed MTBF. Thus, for all of the models discussed in Chapter 3, this logic holds, and suggests the same  $\frac{1}{3}$  rollback factor. We further explore this in Section 5.4, where we search for ideal rollback factors empirically. As we will see, the values we find do not match 0.33 exactly, but this theoretical optimal is indeed effective.

## Chapter 5

### Experimental Results

#### 5.1 Setup

We implemented simulations of CHORE and its variants for a variety of reasons: to re-derive the results of Sigdel et al.; to validate our equations for expected runtimes in various scenarios; and to measure the effectiveness of the Rollback algorithm, which cannot be mathematically modeled in a simple way.

These event-driven simulations were implemented in Python, according to the assumptions of the models: faults are drawn sequentially from an exponential distribution with a fixed mean; checkpointing and recovery require the same duration to complete. Depending on the scenario, faults were either fail-stop or silent; again, depending on the scenario, faults were allowed or prohibited in checkpointing and recovery periods. Comparisons were performed between CHORE variants and the corresponding Young-Daly checkpointing. The fault distributions were seeded, so for any comparison, both models experienced the same faults after the same duration of (fault-possible) runtime. All values found for runtime or overhead were determined from the average of at least 500 runs with different seeds; many scenarios were tested with more. To reduce noise in our results, we will primarily consider long jobs, where the required work is 100 times the MTBF.

#### 5.2 Observations on Sigdel et al.

Beginning with the simulations of Sigdel et al. 's CHORE scenario, we made some interesting observations.

As  $c/\mu$  approaches 0, the overhead of both CHORE and Young-Daly also approach 0, as demonstrated by Figure 5.1.

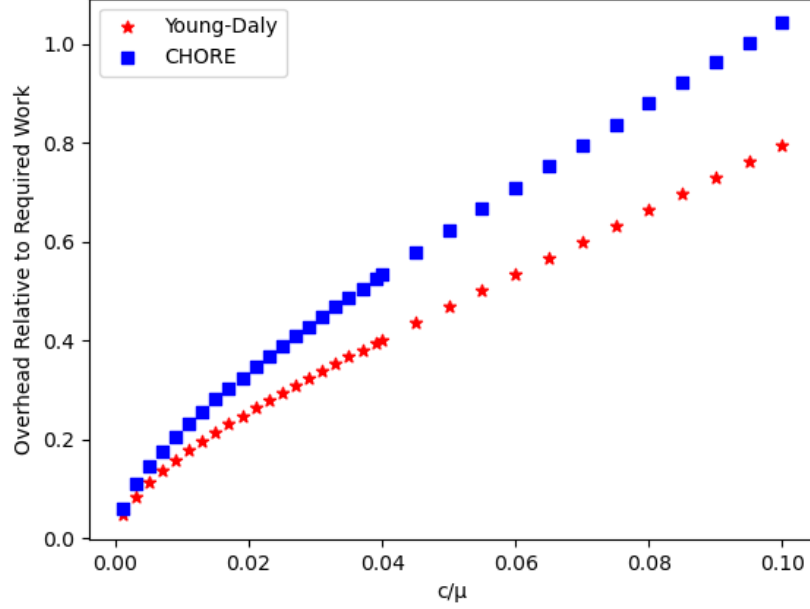


Figure 5.1: Overhead in terms of the minimum required work (simulated;  $w = 100\mu$ ).

When  $c/\mu$  is unreasonably large, CHORE becomes more efficient than the "optimal" Young-Daly checkpointing, as seen in Figure 5.2. This is likely due to the second level of approximation from allowing faults during checkpointing and recovery (with interval size  $\sqrt{2\mu c} - c$  instead of  $\sqrt{2\mu c}$ , as mentioned in Section 2.1.)

### 5.3 Runtime Scenarios

We performed experiments on the three scenarios discussed in Section 3: the original CHORE assumptions, which allows faults to happen during checkpointing and recovery, as in Section 3.2; the original Young-Daly assumptions, where faults are not possible in checkpointing or recovery, as in Section 3.3; and for silent errors (without fail-stop errors), again under the assumption no faults happen during checkpointing or recovery, as in Section 3.4.

Beginning with Sigdel et al.'s assumptions, we attempted to re-derive their maximum ratio of

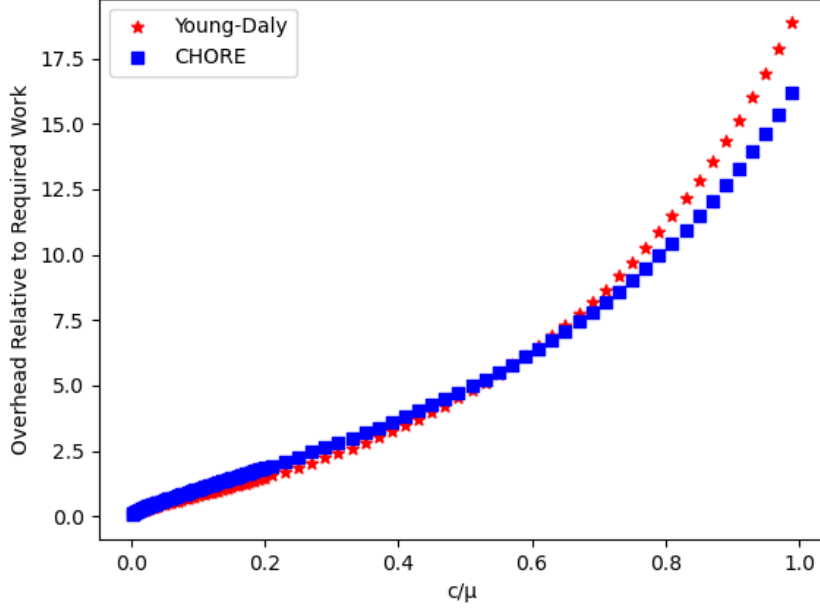


Figure 5.2: Extension of Figure 5.1, CHORE and Young-Daly efficiency for large  $c/\mu$ .

1.26 for CHORE overhead to Young-Daly overhead. In contrast, we found this ratio to be just under 1.331, as shown in Figure 5.3. There are several potential causes for this discrepancy: in [8], the authors describe the 1.26 value as the maximum "checkpointing and rework time overhead" compared to Young-Daly; this could be interpreted as excluding recovery time from overhead. This, however, exacerbates the issue; under this definition, the ratio of  $\frac{\text{CHORE Recovery} + \text{CHORE Lost Work}}{\text{YD Recovery} + \text{YD Lost Work}}$  approaches a maximum of just above 1.37. The most fitting explanation we could find is that the authors are potentially using a different basis; we observed that for what we call the overhead *percentage* ratio, or  $\frac{\text{CHORE OH}}{\text{CHORE Runtime}} / \frac{\text{YD OH}}{\text{YD Runtime}}$ , our simulations give a maximum value of approximately 1.257, which is more consistent with the authors' claim. Henceforth using the definition  $\text{Overhead} = \text{Total Checkpointing} + \text{Total Recovery} + \text{Total Lost Work}$ , we found the maximum ratio of CHORE overhead to Young-Daly overhead to be below 1.34, both from simulation and expected runtime; this maximum value occurred when  $c/\mu = 0.05$ .

Similar analysis was performed on the next scenario, allowing fail-stop faults, but no faults during checkpointing or recovery. With these assumptions, we found the ratio of CHORE overhead

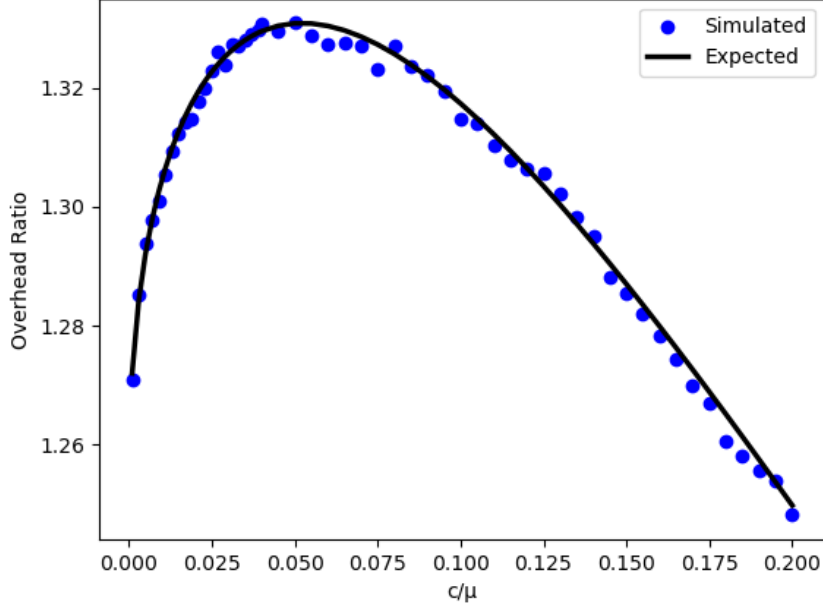


Figure 5.3: CHORE overhead / Young-Daly overhead under Sigdel et al.’s assumptions (fail-stop faults; faults possible during checkpointing and recovery).

to Young-Daly overhead to be lower; specifically, we found that the maximum value this ratio reached was below 1.28, occurring at  $c/\mu = 0.005$  (simulated) and 0.007 (expected). This is demonstrated in Figure 5.4.

Our final scenario, silent errors with no faults during checkpointing or recovery, found similar results. In fact, the ratio of CHORE overhead to Young-Daly overhead was almost identical, again below 1.28. However, under these assumptions, this maximum occurs at  $c/\mu = 0.025$  (simulated) and 0.023 (expected), as Figure 5.5 shows.

## 5.4 Rollback Strategies

Our experiments also included analysis of our novel Rollback variant. Under all three scenarios, we performed a broad-spectrum search of values for our rollback ratio, ranging from 0.03 to 0.90 with a step of 0.03. We limited the range of  $c \leq 10\%$  of  $\mu$ , (with a step of 0.002%) which we expect to encompass all values one is likely to encounter in practice. For each pairing of a rollback

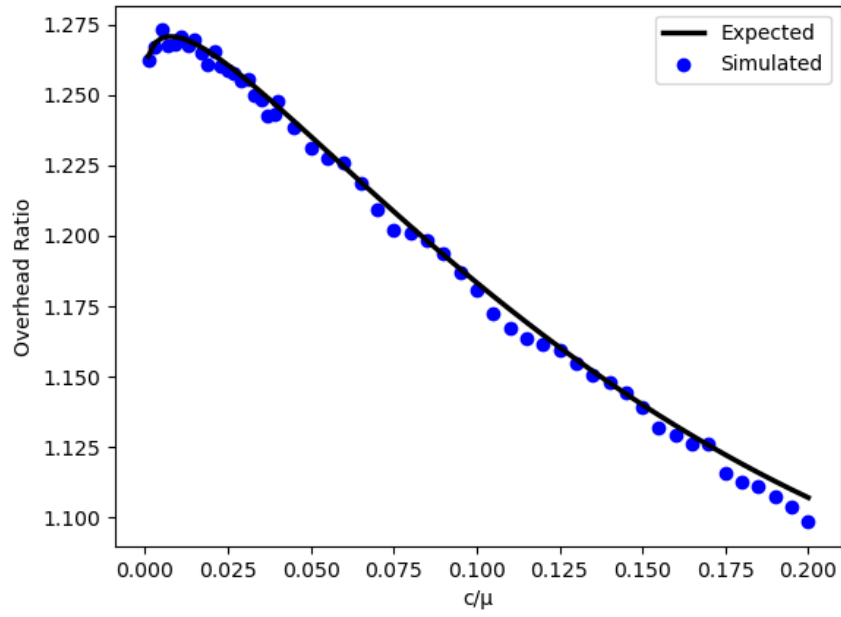


Figure 5.4: CHORE overhead / Young-Daly overhead for fail-stop faults, no faults during checkpointing or recovery.

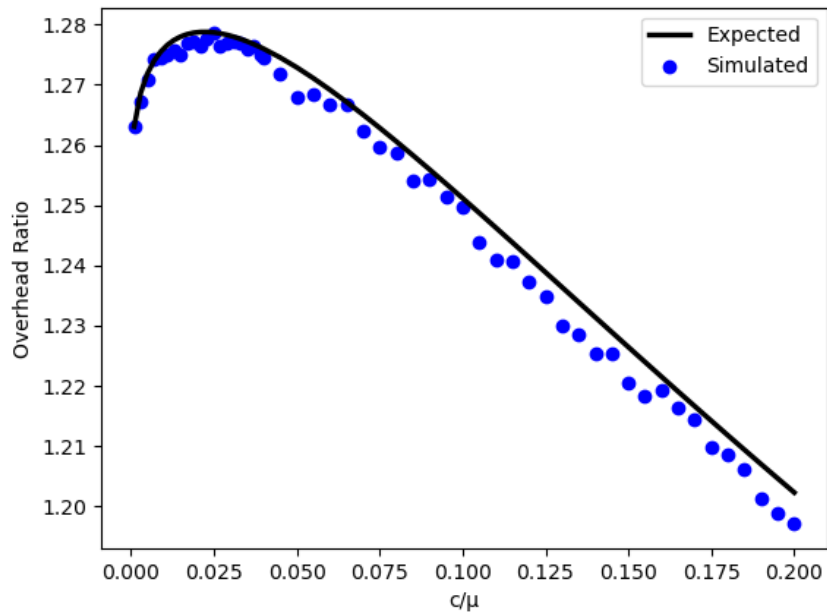


Figure 5.5: CHORE overhead / Young-Daly overhead for silent faults, no faults during checkpointing or recovery.

ratio and a  $c/\mu$  value, 500 runs with required work equal to  $100\mu$  were conducted, and the average runtime was calculated.

Besides the continuous interval growth model discussed in Section 4.2, there were a few other options we considered during generalization. Consider CHORE’s interval 0, which exists only to bootstrap the process by providing an initial quantity of work and checkpointing. This gives intervals 0 and 1 the same size. After a rollback, should the first two intervals be the same size? Ultimately, we decided no. While the first two intervals in a job follow the initial CHORE sequence, we do not repeat intervals sizes after a rollback for several reasons. First, it simplifies the derivation of a theoretical optimal ratio we presented previously. Second, it has very little impact for realistic values of  $c/\mu$ . When  $c/\mu$  is very large, growing just one or two times may exceed the optimal length of an interval, and repeating the first interval may make a difference; however, in practice, we expect  $c \ll \mu$ .

In a similar vein, we considered a minimum interval size. Of course, in base CHORE, the smallest an interval will be is  $2c$ , but rollback has the potential to go smaller. Again, this turned out not to affect the efficiency of rollback in realistic scenarios. Again, large  $c/\mu$  can quickly lead to surpassing the optimal interval size, and a rollback to a smaller starting interval can help with this. However, it only has an impact in these unrealistic scenarios—recall our motivation for the rollback strategy was noticing that the starting interval size is *too small*, so starting at an even smaller duration is counterproductive, except for the rare case where two or more faults happen almost immediately. For this reason, we allowed our rollback simulations to produce an interval size smaller than the base CHORE algorithm would use, but this difference was negligible under realistic values of  $c/\mu$ .

While various combinations of these options were examined, we present here the findings using interpolated interval growth, no minimum interval size, and no repetition of intervals after a rollback. Alternative assumptions do affect the shape of the effectiveness distribution showcased in the following figures, but do not significantly move the optimal ratios.

Beginning with the assumptions of Sigdel et al., we have Figure 5.6. Of all the scenarios

considered, the rollback strategy provided the most benefit here, reducing the overhead by nearly 15% for the optimal rollback factors. However, while a rollback factor of 0.33 did indeed show a reduction in overhead, the optimal rollback factor for this scenario was closer to 0.45 for very small  $c/\mu$  and approximately 0.6 when  $c/\mu$  approached 0.1. While the effect in this figure is subtle, note that the effectiveness of rollback ratios changes with  $c/\mu$ . This is perhaps most visible in the top-left corner, where a rollback factor of 0.8 is less efficient than the base CHORE algorithm for very small  $c/\mu$ , but quickly shows comparable performance.

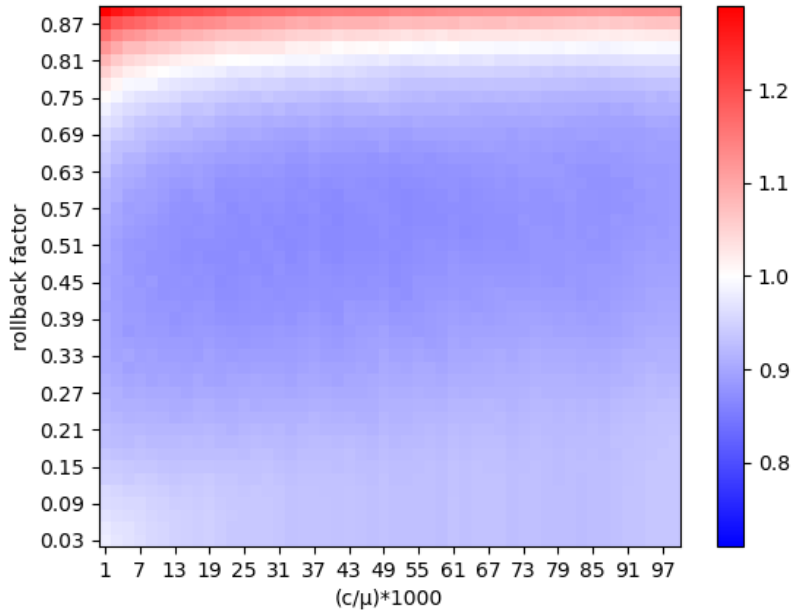


Figure 5.6: Overhead relative to baseline CHORE for various Rollback coefficients; fail-stop faults, faults possible during checkpointing and recovery.

In the next scenario, when we have fail-stop faults but assume checkpointing and recovery are fault-free, the range of effective rollback coefficients is more constrained. This is visualized in Figure 5.7. Here, the theoretically optimal 0.33 is much closer to the experimental ideal, although still perhaps a bit low. Again, we observe the effectiveness of a coefficient depends on  $c/\mu$ , with a number of values showing a reduction in overhead for very small  $c/\mu$  but increasing overhead as this value gets sufficiently large. Fortunately, for a well-selected rollback factor, we expect using the Rollback strategy to at worst perform equivalently to the baseline CHORE algorithm for

realistic values of  $c/\mu$ , and at best, to reduce overhead by approximately 10%

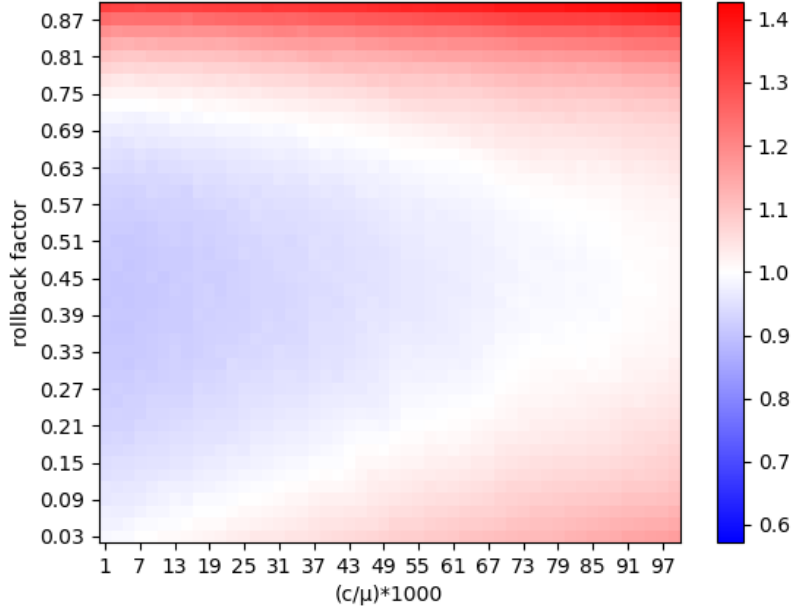


Figure 5.7: Overhead relative to CHORE for various Rollback coefficients; fail-stop faults, no faults during checkpointing or recovery.

Our final scenario, as shown in Figure 5.8, considers only silent faults that cannot occur during checkpointing or recovery. We find a similar distribution to our previous scenario. However, in this case, rollback coefficients appear to remain beneficial for a larger range of  $c/\mu$  values; again, 0.33 is an effective rollback factor, if perhaps slightly lower than ideal. As with the previous scenario, we find overhead to be reduced by approximately 10% for a well-selected factor.

Across all models, we find that the theoretically optimal rollback factor of 0.33 to be effective, if not completely optimal. In general, a factor of 0.45 seems to perform better across all scenarios, although the exact value of  $c/\mu$  does influence the ideal coefficient. This implies that the approximations made during the derivation of the optimal rollback factor may be larger than desired. Indeed, as noted in [8], one observes that checkpointing too soon— that is, after an interval  $n$  units smaller than the optimal duration— incurs less overhead than checkpointing too late, with an interval size  $n$  units larger than optimal. In other words, it is better to err on the side of checkpointing too frequently, which would lead us to expect the optimal value in simulation to be *lower*

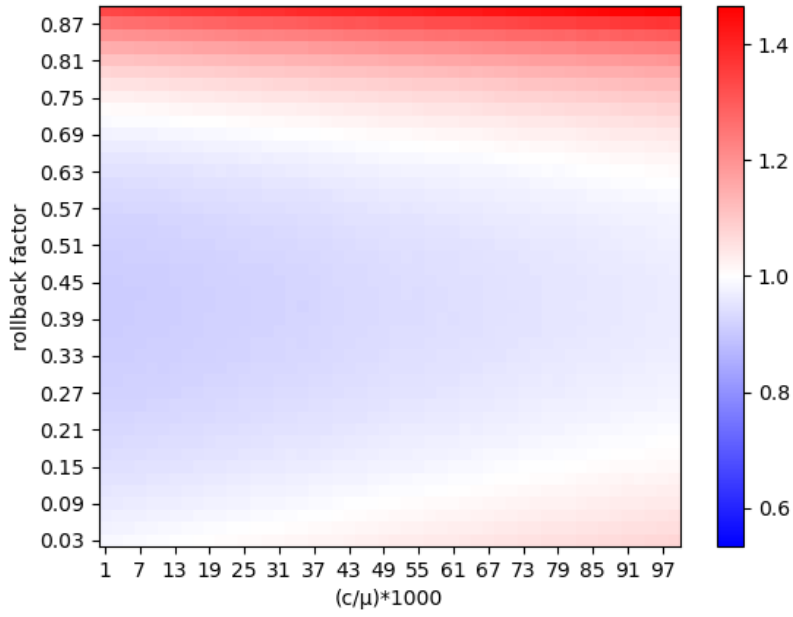


Figure 5.8: Overhead relative to CHORE for various Rollback coefficients; silent faults, no faults during checkpointing or recovery.

than the theoretical optimal: in practice, faults will not fall exactly as expected, and using a smaller rollback factor (and thus returning to a smaller interval, or "erring on the side of caution") should perform better than the alternative. Nonetheless, our theoretical optimal serves as a reasonable approximation, and a starting point for fine-tuning this parameter.

## Chapter 6

### Conclusion

#### 6.1 Findings

In this work, we have analyzed the CHORE checkpointing model presented in [8] under the author’s initial assumptions, and expanded their work to consider restrictions on fault opportunities as well as expanding their model to silent errors. We calculate overhead ratios in a similar manner to [8], and find for fail-stop faults possible at any point in execution, CHORE incurs no more than  $1.34\times$  the overhead of Young-Daly checkpointing; for fail-stop faults assumed not to occur during checkpointing or recovery, CHORE overhead is no more than  $1.28\times$  Young-Daly overhead; and for silent errors assumed not to occur during checkpointing or recovery, CHORE adapted for silent errors is once more limited to  $1.28\times$  the overhead of Young-Daly for silent errors. These values were determined both empirically, through event-driven simulations, and from a theoretical model based on the underlying fault distribution. Our theoretical models have the additional benefit of allowing one to calculate the expected runtime of a job under any of these models for a given quantity of work, checkpointing cost, and MTBF.

Furthermore, we generalize the CHORE algorithm in the context of an additive-increase multiplicative-decrease procedure, and by tuning the decrease, make improvements that reduce overhead by as much as 15% under a wide range of realistic values for checkpointing costs. Furthermore, the justification of our proposed Rollback variant holds under all sets of assumptions considered in this analysis. While optimal rollback factors are impacted by the exact scenario considered, we suggest a number of context-specific values, as well as a general-purpose rollback coefficient derived from a theoretical model.

## 6.2 Future Work

There are several plausible opportunities for future work. One immediate possibility is a mathematical model of the rollback strategy, so the expected runtime of a job and its efficiency relative to CHORE or Young-Daly checkpointing can be directly calculated. Another direction is developing a more realistic model of CHORE and/or Rollback by permitting both fail-stop and silent errors, as both may occur in practice. There is also room for exploration with the more adaptive En-CHORE model presented in [8], which also may benefit from our rollback heuristic. Finally, as one purported benefit of CHORE is that it can out-perform Young-Daly when the standard checkpointing assumptions are not met, it may be worth examining CHORE, Rollback, and Young-Daly checkpointing under different fault distributions, such as a Weibull distribution, or arrivals from an exponential distribution with a variable mean.

## References

- [1] A. Benoit, A. Cavelan, Y. Robert, and H. Sun. Assessing general-purpose algorithms to cope with fail-stop and silent errors. *ACM Trans. Parallel Comput.*, 3(2):1–36, 2016.
- [2] J. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.
- [3] N. El-Sayed and B. Schroeder. Checkpoint/restart in practice: When ‘simple is better’ . In *2014 IEEE International Conference On Cluster Computing (CLUSTER)*, pages 84–92, Los Alamitos, CA, USA, Sept. 2014. IEEE Computer Society.
- [4] N. El-Sayed and B. Schroeder. Understanding practical tradeoffs in hpc checkpoint-scheduling policies. *IEEE Transactions on Dependable and Secure Computing*, 15(2):336–350, 2018.
- [5] Iyer, Butner, and McCluskey. A statistical failure/load relationship: Results of a multicomputer study. *IEEE Transactions on Computers*, C-31(7):697–706, 1982.
- [6] B. Schroeder and G. Gibson. A large-scale study of failures in high-performance computing systems. In *International Conference on Dependable Systems and Networks (DSN’06)*, pages 249–258, 2006.
- [7] B. Schroeder and G. A. Gibson. Understanding disk failure rates: What does an mttf of 1,000,000 hours mean to you? *ACM Trans. Storage*, 3(3):8–es, Oct. 2007.
- [8] P. Sigdel, X. Yuan, and N.-F. Tzeng. Realizing best checkpointing control in computing systems. *IEEE Trans. Parallel Distrib. Syst.*, 32(2):315–329, Feb. 2021.

- [9] D. Tiwari, S. Gupta, G. Gallarno, J. Rogers, and D. Maxwell. Reliability lessons learned from gpu experience with the titan supercomputer at oak ridge leadership computing facility. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '15, New York, NY, USA, 2015. Association for Computing Machinery.
- [10] D. Tiwari, S. Gupta, and S. S. Vazhkudai. Lazy checkpointing: Exploiting temporal locality in failures to mitigate checkpointing overheads on extreme-scale systems. In *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 25–36, 2014.
- [11] J. W. Young. A first order approximation to the optimum checkpoint interval. *Commun. ACM*, 17(9):530–531, Sept. 1974.

# Appendix A

## Expected Runtime Derivation

### A.1 Preliminaries

The following sections show the derivation of expected loss per fault  $E[\bar{L}]$ , expected checkpointing per fault  $E[\bar{C}]$ , expected restoration per fault  $E[\bar{R}]$ , and where applicable, the number of fault occurrences ( $F$ ) per fault detected ( $D$ ),  $E[\frac{F}{D}]$ ; the inverse of this,  $E[\frac{D}{F}]$ , is the term of interest in our equations. As described in Chapter 3, these values can be used to calculate the expected runtime and overhead for each scenario.

Before we begin, we remind readers of the following identities on an exponential distribution with mean  $\mu$ :

$$P(a < x < b) = e^{-a/\mu} - e^{-b/\mu}$$

$$P(a < x) = e^{-a/\mu}$$

$$P(x < b) = 1 - e^{-b/\mu}$$

$$E[x|a < x < b] = \frac{(-be^{-b/\mu} - \mu e^{-b/\mu}) - (-ae^{-a/\mu} - \mu e^{-a/\mu})}{e^{-a/\mu} - e^{-b/\mu}}$$

Some portions of the following derivations are left in terms of probabilities and expected values for simplicity; applying these identities gives the equations specified in Chapter 3.

### A.2 Fail-Stop Faults, Faults During Checkpointing and Recovery

For our first scenario, we consider fail-stop faults, with faults permitted during checkpointing and recovery. Notice that in this scenario (and indeed the ones that follow), the  $\bar{R}$  portion of the equation

will not change from model to model, as we have a fixed restoration time and a restoration must be attempted after each fault. In this scenario,  $E[\bar{R}]$  is simply the odds of completing the restoration, so for both models,

$$E[\bar{R}] = P(r < x) = (1 - e^{-r/\mu})$$

### A.2.1 Young-Daly

Recall that under the Young-Daly model, we calculate an interval of duration  $\sqrt{2\mu c}$  and attempt to run for that period before checkpointing. When there is a possibility of a fault during checkpointing, we work for  $\sqrt{2\mu c} - c$  and spend the last  $c$  creating the checkpoint.

A fault may happen during a recovery period, with the expected loss being  $P(x < r) * E[x|x < r]$ . If we survive the recovery period, loss will happen during an interval. Since an exponential distribution is memoryless, the expected loss (given the fault occurs in this interval) is the same for all intervals. Thus, if we survive restoration, the expected loss is  $E[x|x < \sqrt{2\mu c}]$ . Combining these two cases gives

$$E[\bar{L}_{YD}] = P(x < r) \cdot E[x|x < r] + P(r < x) \cdot E[x|x < \sqrt{2\mu c}]$$

Under this model, checkpoints are made periodically until a fault occurs. Since each interval we survive means another checkpoint is completed, the expected number of checkpoints made (given we survive recovery) is

$$P(\sqrt{2\mu c} < x) + P(2\sqrt{2\mu c} < x) + P(3\sqrt{2\mu c} < x) + \dots$$

Since  $P(a < X) = e^{-a/\mu}$ , this sum becomes a geometric series:

$$e^{-\sqrt{2\mu c}/\mu} + e^{-2\sqrt{2\mu c}/\mu} + e^{-3\sqrt{2\mu c}/\mu} + \dots = e^{-\sqrt{2c/\mu}} \frac{1}{1 - e^{-\sqrt{2c/\mu}}}$$

We can then write the expected checkpointing per fault as

$$E[\bar{C}_{YD}] = P(x > r) \cdot e^{-\sqrt{2c/\mu}} \cdot \frac{1}{1 - e^{-\sqrt{2c/\mu}}} \cdot c$$

which is the final component needed to estimate the duration.

## A.2.2 CHORE

Recall that in this scenario, interval  $n$  under CHORE has duration following

$$dur(n) = \begin{cases} n = 0 & 2c \\ n > 0 & 2nc \end{cases}$$

Also observe that, at the end of interval  $n$ ,

$$\text{Total Runtime} = c(n^2 + n + 2)$$

As interval 0 takes  $2c$  time and the following intervals take  $2c(1 + 2 + \dots + n) = 2c \frac{(n^2 + n)}{2}$  time to complete.

Let us begin deriving our expected loss. As with the previous model, there is the possibility of loss during recovery, with expected loss  $P(x < r) \cdot E[x|x < r]$ . Otherwise, the expected loss under CHORE depends on the size of the interval and the odds of reaching the interval. Assuming we survive recovery, we can write our expected loss as:

$$\begin{aligned} E[\bar{L}_{CH}] &= P(x < 2c) \cdot E[x|x < 2c] + \\ &\quad P(2c < x < 4c) \cdot (E[x|2c < x < 4c] - 2c) + \\ &\quad P(4c < x < 8c) \cdot (E[x|4c < x < 8c] - 4c) + \\ &\quad P(8c < x < 14c) \cdot (E[x|8c < x < 14c] - 8c) + \\ &\quad P(14c < x < 22c) \cdot (E[x|14c < x < 22c] - 14c) + \dots \end{aligned}$$

If we distribute, and move the subtraction to the end, we get:

$$\begin{aligned}
E[\bar{L}_{CH}] &= P(x < 2c) \cdot E[x|x < 2c] + \\
&\quad P(2c < x < 4c) \cdot E[x|2c < x < 4c] + \\
&\quad P(4c < x < 8c) \cdot E[x|4c < x < 8c] + \\
&\quad P(8c < x < 14c) \cdot E[x|8c < x < 14c] + \\
&\quad P(14c < x < 22c) \cdot E[x|14c < x < 22c] + \dots \\
&- [P(2c < x < 4c) \cdot 2c + \\
&\quad P(4c < x < 8c) \cdot 4c + \\
&\quad P(8c < x < 14c) \cdot 8c + \\
&\quad P(14c < x < 22c) \cdot 14c + \dots]
\end{aligned}$$

The first half of the equation is, by definition, the expected value of the exponential distribution, that is,  $\mu$ .

Let us expand the second half of the equation with the identity for  $P(a < x < b)$ . Then the value we subtract is

$$\begin{aligned}
&(e^{-2c/\mu} - e^{-4c/\mu}) \cdot 2c + \\
&(e^{-4c/\mu} - e^{-8c/\mu}) \cdot 4c + \\
&(e^{-8c/\mu} - e^{-14c/\mu}) \cdot 8c + \\
&(e^{-14c/\mu} - e^{-22c/\mu}) \cdot 14c + \dots
\end{aligned}$$

By combining like terms, we get

$$2ce^{-2c/m} + 2ce^{-4c/m} + 4ce^{-8c/m} + 6ce^{-14c/m} + 8ce^{-22c/m} + \dots$$

Recognizing the terms in the exponents follow our equation for total runtime, we can re-write this as

$$2ce^{-2c/m} + \sum_{n=1}^{\infty} 2nc * e^{-c(n^2+n+2)/\mu}$$

Finally, we can write our expected loss as

$$E[\bar{L}_{CH}] = P(x < r) \cdot E[x|x < r] + P(r < x) \cdot (\mu - (2ce^{-2c/m} + \sum_{n=1}^{\infty} 2nc \cdot e^{-c(n^2+n+2)/\mu}))$$

We also need the expected checkpoints per fault to estimate the duration. Fortunately, this is simpler to find. Assuming we survive recovery, if we complete one interval, we make one checkpoint; if we complete the second, we make another, and so on. Thus, the expected number of checkpoints is

$$P(2c < x) + P(4c < x) + P(8c < x) + P(14c < x) + \dots$$

Again, the coefficients correspond to total runtime; since  $P(a < x) = e^{-a/\mu}$ , this series equals  $\sum_{n=0}^{\infty} e^{-c(n^2+n+2)/\mu}$ . Accounting for the possibility of failure during recovery, we can then write the expected successful checkpointing per fault as:

$$E[\bar{C}_{CH}] = P(x > r) \cdot c \cdot \sum_{n=0}^{\infty} e^{-c(n^2+n+2)/\mu}$$

which is the last piece needed to calculate the expected runtime and overhead.

### A.3 Fail-Stop Faults, Safe Checkpointing and Recovery

As with our previous scenario, the expected restoration is equivalent for both models. Since once restoration starts, it always succeeds,

$$E[\bar{R}] = R$$

### A.3.1 Young-Daly

Under this scenario, we no longer have to consider faults during checkpointing or recovery. Thus, faults will always happen during a period of work; each segment of work takes  $\sqrt{2\mu c}$  units of time. Thus, by the memorylessness property of the exponential distribution, the expected loss per fault is simply

$$E[\bar{L}_{YD}] = E[x|x < \sqrt{2\mu c}]$$

Similarly, checkpointing no longer needs to worry about surviving recovery. Since we only have to consider the fault-possible time of an interval (as once a checkpoint starts, it's guaranteed to succeed), we have the same exposure of  $\sqrt{2\mu c}$  before each checkpoint, giving

$$E[\bar{C}_{YD}] = e^{-\sqrt{2c/\mu}} * \frac{1}{1 - e^{-\sqrt{2c/\mu}}} * c$$

### A.3.2 CHORE

As we notice with the Young-Daly checkpointing, in this scenario, we have increased the work in each interval by  $1c$  so the exposure for each interval remains the same as the previous scenario. This is primarily reflected in the equation for total runtime; for our expected loss, the only difference is that we no longer need to account for the possibility of a loss during restoration. This gives us

$$E[\bar{L}_{CH}] = \mu - (2ce^{-2c/\mu} + \sum_{n=1}^{\infty} 2nc \cdot e^{-c(n^2+n+2)/\mu})$$

Similarly, for expected checkpointing, we no longer have to worry about surviving recovery. Although there is no possibility of a fault while making a checkpoint, since our work per interval increases, we have the same odds of surviving each interval, giving:

$$E[\bar{C}_{CH}] = P(x > r) \cdot c \cdot \sum_0^{\infty} e^{-c(x^2+x+2)/\mu}$$

## A.4 Silent Faults, Safe Checkpointing and Recovery

This scenario once more assumes that there are no faults during checkpointing or recovery. Once more, we find our recovery to be

$$E[\bar{R}] = R$$

which holds for both models.

### A.4.1 Young-Daly

For silent errors, Young-Daly checkpointing has a different interval size, doing work for  $\sqrt{\mu c}$  time, then checkpointing. Since, under our assumptions, we only detect silent errors at the end of the interval, the loss is simply the entire interval, or

$$E[\bar{L}_{YD}] = \sqrt{\mu c}$$

As in previous sections, the memorylessness property of the exponential distribution lets us write the expected number of reached checkpoints as a geometric series. Once an interval is started, the odds of reaching the checkpoint without a fault occurring is  $e^{-\sqrt{c/\mu}}$ , then our checkpoint for  $c$  units of time begins. Thus,

$$\begin{aligned} E[\bar{C}_{YD}] &= c \cdot (P(\sqrt{\mu c} < x) + P(2\sqrt{\mu c} < x) + P(3\sqrt{\mu c} < x) + \dots) \\ &= c \cdot e^{-\sqrt{c/\mu}} \cdot \sum_{n=0}^{\infty} e^{-n\sqrt{c/\mu}} \\ &= c \cdot e^{-\sqrt{c/\mu}} \cdot \frac{1}{1 - e^{-\sqrt{c/\mu}}} \end{aligned}$$

To find the faults per fault detected, recall our helper function from Chapter 3,  $\lambda(x)$ , which is the expected number of faults during period  $x$  given that at least one fault occurs.

$$\lambda(x) = \frac{x/\mu}{1 - e^{-x/\mu}}$$

Then the faults per fault detected is simply

$$E_{YD} \left[ \frac{F}{D} \right] = \lambda(\sqrt{\mu c})$$

## A.4.2 CHORE

Recall that for silent errors, CHORE has interval sizes  $2c, 2c, 3c, 4c, \dots$ ; that is, intervals 0 and 1 have duration  $2c$  (including the checkpoint) and the duration of all subsequent intervals increases by  $c$ . As with Young-Daly checkpointing, a fault means losing the entire interval. We can then write our expected loss as

$$\begin{aligned} E[\bar{L}_{CH}] &= c \cdot P(0 < x < c) + c \cdot P(c < x < 2c) + 2c \cdot P(2c < x < 4c) + 3c \cdot P(4c < x < 7c) + \dots \\ &= c(1 - e^{-c/\mu} + e^{-c/\mu} - e^{-2c/\mu} + 2e^{-2c/\mu} - 2e^{-4c/\mu} + 3e^{-4c/\mu} - 4e^{-7c/\mu} + \dots) \\ &= c(1 - e^{-c/\mu}) + c(e^{-c/\mu} - e^{-2c/\mu} + 2e^{-2c/\mu} - 2e^{-4c/\mu} + 3e^{-4c/\mu} - 4e^{-7c/\mu} + \dots) \\ &= c(1 - e^{-c/\mu}) + c(e^{-c/\mu} + e^{-2c/\mu} + e^{-4c/\mu} + e^{-7c/\mu} + \dots) \\ &= c(1 - e^{-c/\mu}) + c\left(\sum_{n=0}^{\infty} e^{-(\frac{n^2+n}{2}+1)c/\mu}\right) \end{aligned}$$

Similarly to our initial scenario, we notice that the total (fault-possible) runtime after interval  $n$  is  $c(\frac{n^2+n}{2} + 1)$ ; this is reflected in the coefficient of the power in the sum.

Our expected checkpointing follows a similar pattern. For the first checkpoint, we must survive  $1c$  of work; for the second, we must have survived the first  $1c$  of work and an additional  $1c$  for the second interval, and so on; to create checkpoint number  $n$ , we must have survived the past  $n$  intervals worth of work. This gives us

$$\begin{aligned} E[\bar{C}_{CH}] &= c(P(c < x) + P(2c < x) + P(4c < x) + P(7c < x) + \dots) \\ &= c * \sum_{n=0}^{\infty} P((\frac{n^2+n}{2} + 1)c < x) \\ &= c * \sum_{n=0}^{\infty} e^{-(\frac{n^2+n}{2}+1)c/\mu} \end{aligned}$$

Finally we determine the expected faults per fault detected. For each interval, we take the odds a fault occurs times the expected number that of faults, given at least one occurs. Then,

$$\begin{aligned}
 E_{CH} \left[ \frac{F}{D} \right] &= P(0 < x < c) \cdot \lambda(c) + P(c < x < 2c) \cdot \lambda(c) + P(2c < x < 4c) \cdot \lambda(2c) + \\
 &\quad P(7c < x < 11c) \cdot \lambda(4c) + P(4c < x < 7c) \cdot \lambda(3c) + \dots \\
 &= P(0 < x < c) \cdot \lambda(c) + \sum_{n=0}^{\infty} P\left(\frac{n^2+n}{2} + 1 < x < \frac{(n+1)^2+(n+1)}{2} + 1\right) \cdot \lambda((n+1) \cdot c)
 \end{aligned}$$

which is the final term needed for our calculation.

## Appendix B

### Interval Interpolation

#### B.1 Fail-Stop Faults

We will derive a relationship between the runtime of a sequence and the number of completed intervals under CHORE. This derivation relies on the linear rate of growth between intervals; as an interval 0 would break the pattern, we will exclude it for this derivation. In practice, we consider these formulas undefined for interval 0.

We begin with the standard CHORE model, where each successive interval grows by  $2c$ . Let the first interval have duration (in terms of  $c$ )  $zc$ , and we want to relate it to the total runtime (also in terms of  $c$ )  $tc$ . A sequence of  $n$  intervals gives the total runtime

$$\begin{aligned} tc &= zc + (zc + 2c) + (zc + 4c) + \cdots + (zc + 2c(n-1)) \\ &= nzc + (2c + 4c + \cdots + 2c(n-1)) \\ &= nzc + 2c(1 + 2 + 3 \cdots + (n-1)) \end{aligned}$$

With the identity  $1 + 2 + \cdots + x = \frac{x(x+1)}{2}$ , we find

$$\begin{aligned} tc &= nzc + 2c \frac{(n-1+1)(n-1)}{2} \\ &= nzc + c(n^2 - n) \\ &= nzc + cn^2 - cn \\ &= c(n^2 + (z-1)n) \end{aligned}$$

So at the end of interval  $n$ , the total time passed is  $tc$ , given by this equation.

We can cancel a  $c$  on each side of the equation; then applying the quadratic formula, we find that with  $t$  from  $tc$  and  $z$  from  $zc$ , we are at the end of interval number

$$n = \frac{-(z-1) + \sqrt{(z-1)^2 + 4t}}{2}$$

Furthermore, since the first interval has size  $zc$  and each following interval grows by  $2c$ , interval number  $n$  has duration  $zc + (n-1)2c$ .

## B.2 Silent Faults

The derivation for the silent CHORE model follows an almost identical derivation, with the only difference being intervals grow by  $c$  instead of  $2c$ . We then find that

$$tc = c(\frac{1}{2}n^2 + (z - \frac{1}{2})n)$$

Again canceling a  $c$  on each side for simplicity, we find that with  $z$  from  $zc$  and  $t$  from runtime  $tc$ , we are at the end of interval number

$$n = -(z - \frac{1}{2}) + \sqrt{(z - \frac{1}{2})^2 + 2t}$$

Finally, since interval growth is only  $1c$  smaller, interval  $n$  will have duration  $zc + (n-1)c$